

Actividad: Exploración de Escenarios en Redes Neuronales Convolucionales

Materia: Redes Neuronales y SVM

Profesor: Dr. Aboud Barsekh Onji

Tipo de actividad: Práctica experimental con análisis comparativo

Entregable: Reporte en PDF con código, tablas y conclusiones

Objetivo

Explorar el impacto de distintas configuraciones arquitectónicas y de entrenamiento en el desempeño de una red neuronal convolucional (CNN) para clasificación de imágenes, a partir del ejemplo base provisto en MATLAB.

Ejemplo base (referencia)

Se utiliza en esta actividad el dataset DigitsData.zip que contiene imágenes de dígitos escritos a mano. Acuérdese que puede modificar el mismo ejemplo para adaptarlo a sus necesidades. Para abrir el ejemplo ejecute este comando en MATLAB:

```
openExample('nnet/CreateImageClassificationNetworkUsingDeepNetworkDesignerExample')
```

También si no quiere modificar el ejemplo original, puede usar el siguiente código como punto de partida, siempre y cuando crea los scripts en la misma carpeta donde se encuentra el archivo DigitsData.zip; cópialo y crea variantes para cada escenario.

```
% Cargar datos
unzip("DigitsData.zip")
imds = imageDatastore("DigitsData", ...
    IncludeSubfolders=true, ...
    LabelSource="foldernames");
classNames = categories(imds.Labels);

[imdsTrain, imdsValidation, imdsTest] = ...
    splitEachLabel(imds, 0.7, 0.15, 0.15, "randomized");

% Arquitectura base (exportada desde Deep Network Designer)
layers = [
    imageInputLayer([28 28 1])
    convolution2dLayer(3, 8, Padding="same")
```

```

        batchNormalizationLayer
        reluLayer
        fullyConnectedLayer(10)
        softmaxLayer
    ];

    % Opciones de entrenamiento
    options = trainingOptions("sgdm", ...
        MaxEpochs=4, ...
        ValidationData=imdsValidation, ...
        ValidationFrequency=30, ...
        Plots="training-progress", ...
        Metrics="accuracy", ...
        Verbose=false);

    % Entrenamiento y evaluación
    net = trainnet(imdsTrain, layers, "crossentropy", options);
    accuracy = testnet(net, imdsTest, "accuracy");
    fprintf("Precisión en prueba: %.2f%%\n", accuracy)

```

Instrucciones generales

1. Ejecuta el **Escenario 0 (base)** sin modificaciones y registra los resultados.
 2. Para cada escenario posterior, modifica **únicamente** el parámetro indicado y mantén todo lo demás igual al escenario base (puedes hacer las modificaciones en el mismo script, o usar la app ‘Deep Network Designer’ para modificar la arquitectura y luego exportar la red para cada escenario).
 3. Registra para cada escenario: precisión en validación, precisión en prueba y tiempo de entrenamiento aproximado.
 4. Completa la **tabla comparativa** con los resultados obtenidos.
 5. Redacta las **conclusiones** respondiendo las preguntas guía al final.
-

Escenarios

Escenario 0 — Modelo base (sin cambios)

Ejecuta el código tal como aparece en el ejemplo original.

Parámetros fijos: - Filtros en conv2D: 8, tamaño de kernel: 3×3 - Épocas: 4 - Optimizador: `sgdm` - Partición: 70 / 15 / 15

Escenario 1 — Variación del número de filtros convolucionales

Modifica el número de filtros en `convolution2dLayer`. Prueba los siguientes valores:

Variante	Filtros
1-A	4
1-B	16
1-C	32

```
% Ejemplo para variante 1-B  
convolution2dLayer(3, 16, Padding="same")
```

Escenario 2 — Variación del número de épocas

Modifica `MaxEpochs` en las opciones de entrenamiento:

Variante	MaxEpochs
2-A	2
2-B	8
2-C	15

```
options = trainingOptions("sgdm", ...  
    MaxEpochs=8, ...    % cambia este valor  
    ...
```

Escenario 3 — Cambio de optimizador

Sustituye el optimizador en `trainingOptions`:

Variante	Optimizador
3-A	"adam"
3-B	"rmsprop"
3-C	"sgdm" (base, referencia)

```
options = trainingOptions("adam", ...    % cambia el optimizador
```

Escenario 4 — Adición de una segunda capa convolucional

Agrega un bloque convolucional adicional después del primero:

```
layers = [  
    imageInputLayer([28 28 1])  
    convolution2dLayer(3, 8, Padding="same")  
    batchNormalizationLayer  
    reluLayer  
    convolution2dLayer(3, 16, Padding="same") % + nueva capa  
    batchNormalizationLayer % + nueva capa  
    reluLayer % + nueva capa  
    fullyConnectedLayer(10)  
    softmaxLayer  
];
```

Registra si mejora o empeora el resultado respecto al escenario base.

Escenario 5 — Variación de la partición de datos

Modifica la proporción entre entrenamiento, validación y prueba:

Variante	Entrenamiento	Validación	Prueba
5-A	50%	25%	25%
5-B	80%	10%	10%
5-C	70%	15%	15% (base)

```
% Ejemplo para variante 5-A  
[imdsTrain, imdsValidation, imdsTest] = ...  
    splitEachLabel(imds, 0.5, 0.25, 0.25, "randomized");
```

Tabla comparativa de resultados

Completa la siguiente tabla con los valores obtenidos en cada experimento. Incluye esta tabla en tu reporte.

Escenario	Descripción	Acc_train (%)	Acc_val (%)	Acc_test (%)	Tiempo aprox. (s)
0 — Base	8 filtros, 4 épocas, sgdm, 70/15/15				
1-A	4 filtros				
1-B	16 filtros				
1-C	32 filtros				

Escenario	Descripción	Acc_train (%)	Acc_val (%)	Acc_test (%)	Tiempo aprox. (s)
2-A	2 épocas				
2-B	8 épocas				
2-C	15 épocas				
3-A	Optimizador adam				
3-B	Optimizador rmsprop				
4	2 bloques conv				
5-A	Partición 50/25/25				
5-B	Partición 80/10/10				

Preguntas guía para las conclusiones

Responde cada pregunta con base en los datos de tu tabla comparativa. Justifica tus respuestas con evidencia numérica. **Evita usar expresiones cualitativas y variables lingüísticas como “mejor”, “peor”, “más”, “menos”, etc. En su lugar, usa términos precisos y cuantitativos.**

1. **Filtros convolucionales:** ¿Qué ocurre con la precisión al aumentar o disminuir el número de filtros? ¿Existe un punto de rendimiento decreciente? ¿A qué atribuyes ese comportamiento?
2. **Épocas de entrenamiento:** ¿Cómo evoluciona la precisión conforme aumentan las épocas? ¿Observaste algún indicio de sobreajuste (*overfitting*)? ¿Cómo podrías detectarlo en las gráficas de entrenamiento?
3. **Optimizadores:** ¿Cuál optimizador ofreció el mejor desempeño en este dataset? ¿Qué diferencias conceptuales existen entre `sgdm`, `adam` y `rmsprop` que podrían explicar los resultados?
4. **Profundidad de la red:** ¿Agregar una segunda capa convolucional mejoró el desempeño? ¿Por qué crees que sucede (o no sucede) esa mejora en un dataset tan simple como dígitos de 28×28 ?
5. **Partición de datos:** ¿Cómo afecta la cantidad de datos de entrenamiento a la precisión final? ¿Cuál partición consideras más adecuada para este problema y por qué?
6. **Reflexión global:** Si tuvieras que seleccionar la mejor configuración para producción, ¿cuál elegirías considerando precisión, tiempo de entrenamiento y complejidad del modelo? Justifica tu elección.

Criterios de evaluación

Criterio	Ponderación
Correcta ejecución de todos los escenarios	30%
Tabla comparativa completa y coherente	25%
Calidad y profundidad de las conclusiones	30%
Presentación y orden del reporte	15%

Formato del reporte

- **Extensión:** 4–8 páginas
 - **Incluir:** Capturas de pantalla de las gráficas de entrenamiento de al menos 3 escenarios
 - **Formato de entrega:** PDF con el código de cada escenario en apéndice
 - **Nombre del archivo:** NombreApellido_ActividadCNN.pdf
-